

مَسَارَا

M A S A R A

منصة ذكية محوكة لإدارة وتتبع النقل المدرسي – تجمع بين التتبع اللحظي، إدارة الأسطول، وذكاء اصطناعي خاضع للرقابة البشرية في كل قرار تشغيلي.

Gemini AI

محرك الذكاء الاصطناعي

Express + SQLite

خادم وقاعدة بيانات

React 19 + TypeScript

واجهة أمامية

PILOT READY

حالة الجاهزية

فهرس المحتويات

دليل شامل لكل ما يلزم معرفته عن مشروع مَسَارًا – من الفكرة إلى التشغيل والأمان

الفكرة، المشكلة، الفئات المستفيدة

١ نبذة عن المشروع

بوابات الأدوار، الخرائط، الذكاء الاصطناعي، الإشعارات

٢ الميزات الرئيسية

المكدس التقني، النظامان المحوكم والقديم، قاعدة البيانات

٣ البنية التقنية

أهم المجلدات والملفات

٤ هيكل المشروع

المتطلبات، الإعداد، أوامر التشغيل

٥ التشغيل خطوة بخطوة

جدول كامل للأوامر وبيانات الدخول التجريبية

٦ أوامر npm والحسابات التجريبية

نتائج التدقيق الأمني وحالة الجاهزية

٧ الأمان والصلاحيات

استكشاف الأخطاء، أسئلة متكررة

٨ ملاحظات ونصائح مهمة

نبذة عن المشروع



مَسَارًا (MASARA) هي منصة رقمية متكاملة لإدارة وتتبع رحلات النقل المدرسي، مصممة لترتبط أربع فئات في نظام واحد مباشر ومتزامن: **أولياء الأمور، السائقين، إدارة المدرسة، والمشرف العام على المنظومة**. الهدف هو تحويل رحلة الطالب اليومية من عملية غير مرئية إلى تجربة شفافة، آمنة، وقابلة للمتابعة لحظة بلحظة.

المشكلة التي يعالجها المشروع

في النظام التقليدي للنقل المدرسي، لا يملك ولي الأمر أي رؤية حقيقية لموقع الحافلة أو وقت وصولها، ولا تملك المدرسة أداة مركزية لمراقبة الأسطول أو حالة صعود الطلاب، بينما يعتمد السائق على تنسيق يدوي غير موثوق. مَسَارًا يحل هذه الفجوة عبر تتبع مباشر على الخريطة، تحديثات فورية لحالة كل طالب (بالانتظار / صعد / وصل / غائب)، وتنبيهات تلقائية قبل وصول الحافلة بدقائق معدودة.

الفئات المستفيدة

السائق



بدء الرحلة، تسجيل صعود كل طالب عند نقطة التوقف، والإبلاغ الفوري عن أي طارئ في المسار.

ولي الأمر



متابعة موقع حافلة ابنه لحظياً، معرفة الوقت المتوقع للوصول، تسجيل غياب، واستقبال تنبيهات قبل الوصول بخمس دقائق.

المشرف العام



إدارة كاملة للأسطول والمسارات والطلاب، مراجعة توصيات الذكاء الاصطناعي، والموافقة على الإجراءات التشغيلية.

المدرسة



لوحة تحكم شاملة لحالة جميع الطلاب والحافلات في وقت واحد، مع تقارير حضور فورية.

ملاحظة تعريفية

اسم المشروع "مَسَارًا" مستوحى من كلمة "المسار"، في إشارة إلى الطريق الذكي والآمن الذي ترسمه المنصة لكل رحلة مدرسية يومية.

الميزات الرئيسية

بوابات مخصصة لكل دور

يعرض التطبيق واجهة مختلفة تماماً حسب دور المستخدم بعد تسجيل الدخول، بحيث يرى كل طرف فقط ما يحتاجه:

- **بوابة ولي الأمر (Parent Portal):** خريطة حية لموقع حافلة الأبناء، حالة كل طفل، جدولة تنبيهات الوصول، وزر الإبلاغ عن غياب.
- **بوابة السائق (Driver Portal):** قائمة الطلاب في الحافلة المخصصة له، أزرار بدء الرحلة وتسجيل الصعود، والإبلاغ عن حوادث الطريق.
- **لوحة المدرسة (School Dashboard):** نظرة شاملة على جميع الطلاب والحافلات التابعة للمدرسة مع مؤشرات الحضور والغياب.
- **بوابة وكيل الذكاء الاصطناعي الإداري (Admin AI Agent Portal):** تشغيل وكلاء الذكاء الاصطناعي لتحسين المسارات، كشف التحويلات الطارئة، والتنبؤ بأوقات الوصول.

الخريطة الحية والتتبع

عرض تفاعلي (Leaflet) لمواقع الحافلات، نقاط التوقف، والمسارات على خريطة مسقط، مع تحديث دوري لحالة كل حافلة (السرعة، نسبة الإشغال، المحطة القادمة، الوقت المتوقع للوصول).

منظومة الذكاء الاصطناعي المحكم

هذا هو العمود الفقري المتقدم في مسارا: أي اقتراح يصدر عن الذكاء الاصطناعي – سواء لتحسين مسار، أو التعامل مع تأخير، أو تنبيه سلامة – **لا يُنقَذ تلقائياً أبداً**. يمر كل اقتراح عبر ثلاث طبقات:

٣. منفذ الإجراءات

ينفذ فقط ما تمت الموافقة عليه فعلياً، ويسجل كل خطوة في سجل تدقيق دائم.

٢. مركز الموافقات

يعرض كل توصية على مسؤول بشري لمراجعتها والموافقة عليها أو رفضها.

١. محرك السياسات

يتحقق أن الاقتراح يطابق القواعد التشغيلية المسموح بها قبل عرضه على أي مسؤول.

لماذا هذا مهم

هذا التصميم يمنع أي قرار آلي غير مُراقب من التأثير على سلامة الطلاب أو بيانات النظام – كل إجراء فعلي وراءه قرار بشري موثّق.

الإشعارات ومركز العمليات

- تنبيهات تلقائية للصعود، الغياب، واقتراب وصول الحافلة.
- مركز المحاكاة (Simulation Center) لتجربة سيناريوهات تشغيلية (تأخير، حادث، رحلة عادية) دون التأثير على البيانات الحقيقية.
- سجل الأحداث التشغيلية (Operations Feed) الذي يعرض كل ما يحدث في النظام لحظة بلحظة.

البنية التقنية

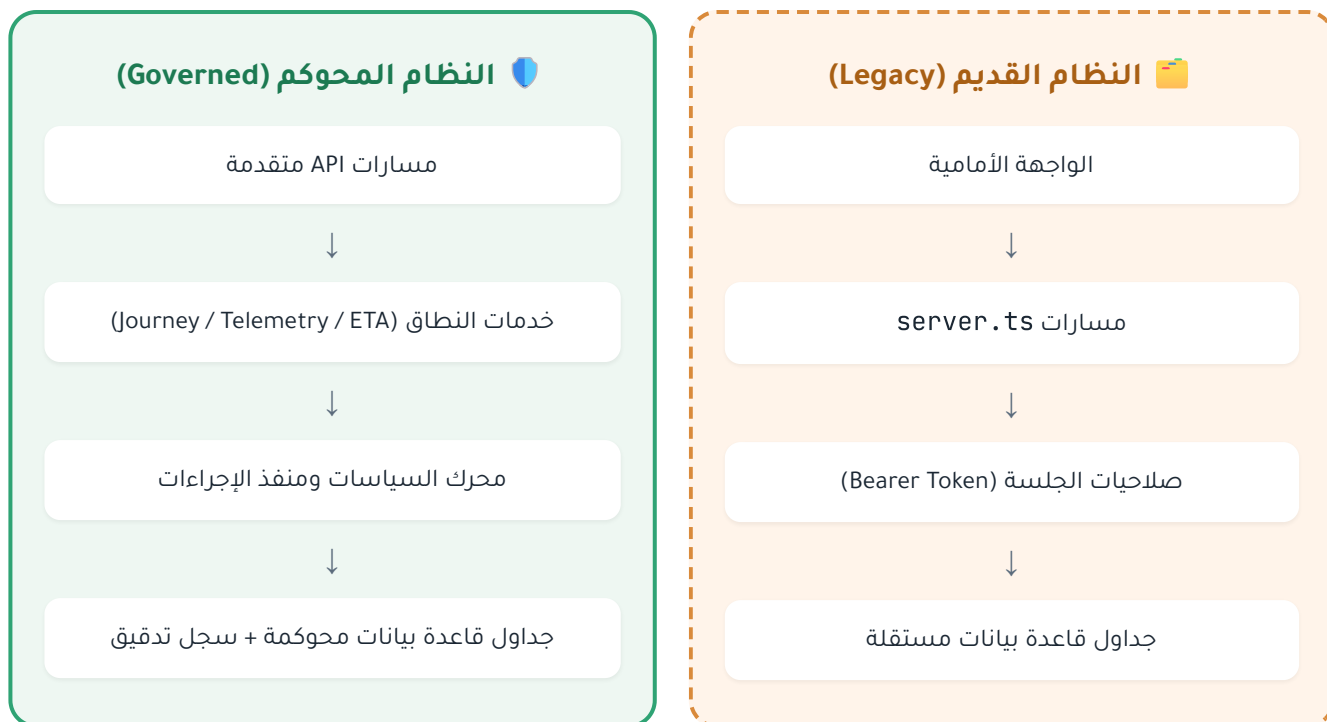
٣

المكدس التقني (Tech Stack)

الطبقة	التقنية	الغرض
الواجهة الأمامية	React 19 + TypeScript + Vite	واجهة مستخدم تفاعلية بدعم كامل للغة العربية (RTL)
الأنماط	Tailwind CSS 4	تصميم متجاوب وسريع البناء
الخرائط	Leaflet	عرض المواقع والمسارات اللحظية
الخادم	Node.js + Express	واجهات برمجية (API) للواجهة الأمامية
قاعدة البيانات	SQLite + Drizzle ORM	تخزين دائم لكل بيانات النظام
الذكاء الاصطناعي	Google Gemini API	توليد التوصيات التشغيلية (تحليل فقط، دون تنفيذ مباشر)
الاختبارات	Vitest	أكثر من 900 اختبار آلي تغطي كل الأنظمة

بنية مزدوجة: النظام المحوكم مقابل النظام القديم

يحتوي المشروع على مسارين منفصلين تماماً ومتعقدين، لا يتقاطعان في قواعد البيانات أو منطق الصلاحيات:



هذا الفصل المتعمد يعني أن النظام الأساسي للحكومة والذكاء الاصطناعي (تتبع الرحلات، الوقت المتوقع للوصول، التوصيات، سجل التدقيق) محمي تماماً من أي تعديل في الواجهات القديمة – أي تحسين يُضاف لاحقاً لا يُمكن أن يكسر ضمانات السلامة والحكومة القائمة.

قاعدة البيانات

تُستخدم SQLite كقاعدة بيانات وحيدة (ملف database/masara.db) عبر Drizzle ORM. مع تفعيل وضعي WAL (كتابة متزامنة آمنة) وforeign_keys (فرض العلاقات بين الجداول). يحتوي المخطط الحالي على أكثر من عشرين جدولاً موزعة بين الجانب المحكوم (المدارس، الحافلات، الطلاب، الرحلات، التتبع، التوصيات، سجل التدقيق) والجانب القديم (المستخدمون، الجلسات، الحافلات، الطلاب القديمة).

٤ هيكل المشروع

أهم المجلدات في جذر المشروع وما تحتويه:

```

masara-مسارًا/
# الواجهة الأمامية
├─ src/
│   ├─ components/      React components (كل بوابة وناظفة)
│   ├─ services/         من الواجهة API استدعاءات
│   ├─ types.ts          تعريفات الأنواع (Bus, Student, Route ...)
│   └─ mockData.ts       بيانات تجريبية أولية للواجهة
# الخادم
├─ server.ts             نقطة الدخول + مسارات النظام القديم
├─ server/
│   ├─ routes/           مسارات (Journey/ETA/Telemetry...)
│   ├─ services/         منطق الأعمال (PolicyEngine, ActionExecutor, ...)
│   ├─ repositories/     طبقة الوصول لقاعدة البيانات
│   └─ middleware/       (الخ, إعدادات الأمان)
# قاعدة البيانات
├─ database/
│   ├─ schema.ts         تعريف كل الجداول
│   ├─ migrations/       المرقمة (SQL) ملفات الترحيل
│   └─ seed/seed.ts      بيانات تجريبية أولية لقاعدة البيانات
# الاختبارات
├─ tests/                أكثر من 900 اختبار آلي
# الإعدادات

```


التشغيل خطوة بخطوة

المتطلبات الأساسية

- **Node.js** (يفضل الإصدار 20 أو أحدث)
- **npm** (يأتي مع Node.js)
- مفتاح **Gemini API** اختياري – النظام يعمل بوضع محاكاة (mock) بدونهِ تماماً

1

تثبيت الحزم

من داخل مجلد المشروع، نَقِّذ:

```
npm install
```

2

إعداد متغيرات البيئة

انسخ ملف `.env.example` إلى ملف جديد باسم `.env` ثم عدّل القيم حسب الحاجة (اتركها كما هي للتجربة المحلية):

```
# .env
GEMINI_API_KEY="ضع_مفتاحك_هنا" # اختياري
DATABASE_URL="./database/masara.db"
AI_MODE="mock" # إذا توفر المفتاح "gemini" أو #
```

3

تجهيز قاعدة البيانات

لإنشاء الجداول وتعبئتها ببيانات تجريبية (مدرسة، حافلات، طلاب، مستخدمون):

```
npm run db:migrate # إنشاء الجداول
npm run db:seed # تعبئة بيانات تجريبية

# أو الاثنان معاً:
npm run db:reset
```

4

تشغيل وضع التطوير

يشغل هذا الأمر الخادم والواجهة معاً على نفس المنفذ مع إعادة التحميل التلقائي عند أي تعديل:

```
npm run dev
```

ثم افتح المتصفح على `http://localhost:3000`

5

البناء والتشغيل بوضع الإنتاج

لإنشاء نسخة إنتاجية محسّنة ثم تشغيلها:

```
npm run build
npm start
```

6

تشغيل الاختبارات

للتأكد من سلامة كل أجزاء النظام (أكثر من 900 اختبار):

```
npm test
```

تنبيه هام

عند إعادة تعبئة قاعدة بيانات محلية مُعبّأة مسبقاً، احذف ملفات قاعدة البيانات القديمة أولاً لتفادي تعارض القيود الفريدة (Unique Constraints):

```
rm -f database/masara.db database/masara.db-shm database/masara.db-wal
npm run db:reset
```

أوامر npm والحسابات التجريبية

جدول أوامر npm الكاملة

الوظيفة	الأمر
تشغيل بيئة التطوير مع إعادة التحميل التلقائي	npm run dev
بناء نسخة إنتاجية للواجهة والخادم	npm run build
تشغيل النسخة الإنتاجية المبنية مسبقاً	npm start
تشغيل جميع الاختبارات الآلية	npm test
فحص الأنواع البرمجية (TypeScript) دون بناء	npm run lint
توليد ملف ترحيل جديد بعد تعديل المخطط	npm run db:generate
تطبيق كل ملفات الترحيل على قاعدة البيانات	npm run db:migrate
تعبئة قاعدة البيانات ببيانات تجريبية	npm run db:seed
ترحيل + تعبئة معاً (الأكثر استخداماً محلياً)	npm run db:reset
حذف مجلدات البناء الناتجة	npm run clean

الحسابات التجريبية الجاهزة

بعد تنفيذ `npm run db:seed` ، تتوفر أربعة حسابات تجريبية جاهزة لتسجيل الدخول فوراً – كلمة المرور واحدة لجميعها:

الدور	البريد الإلكتروني	كلمة المرور
ولي أمر	parent@masara.om	password123
سائق	driver1@masara.om	password123
مدرسة	school@masara.om	password123
مشرف عام (Admin)	admin@masara.om	password123

للتجربة السريعة

تحتوي شاشة تسجيل الدخول على أزرار "دخول سريع تجريبي" لكل دور – تملأ البيانات تلقائياً دون الحاجة لكتابتها يدوياً.

الأمان والصلاحيات

V

خضع المشروع لتدقيق أمني شامل ونهائي (تدقيق الجاهزية الإنتاجية) غطى المصادقة، الصلاحيات، الجلسات، قاعدة البيانات، والتحقق الحي عبر تشغيل نسختين مستقلتين من الخادم في آن واحد. النتيجة:

● جاهز للتجربة التشغيلية (Pilot Ready)

نقاط القوة المؤكدة

الجلسات 🔑

رمز جلسة عشوائي bit-256، يُخزَّن مُجَزَّأً فقط، مع انتهاء صلاحية وإلغاء فوري عند تسجيل الخروج.

كلمات المرور 🔒

تشفير scrypt مع ملح عشوائي لكل كلمة مرور – لا تُخزَّن أو تُعرض أبداً بصيغة واضحة.

ملكية الموارد 👤

كل سائق يتحكم فقط بحافلته، وكل ولي أمر يطلع فقط على بيانات ابنه – مُثبت باختبار حي متعدد العمليات.

تحديد محاولات الدخول 🚦

حظر مؤقت بعد 5 محاولات فاشلة، ويعمل عبر جميع نسخ الخادم وليس نسخة واحدة فقط.

عدم تسرب البيانات الحساسة 📄

تأكيد أن كلمات المرور والرموز لا تظهر أبداً في الاستجابات أو السجلات.

حوكمة الذكاء الاصطناعي 🤖

لا تنفيذ تلقائي لأي توصية دون موافقة بشرية موثقة عبر محرك السياسات.

القيود المعروفة (خارج نطاق الكود البرمجي)

القيود	التصنيف	التفاصيل
لا يوجد مزود بريد/رسائل نصية/إشعارات دفع حقيقي	اعتماد خارجي	البنية جاهزة بالكامل، تنتظر ربط مزود فعلي وبيانات اعتماد حقيقية

القيد	التصنيف	التفاصيل
لا توجد مصادقة ثنائية (MFA)	تحسين مستقبلي	غير مطلوبة إلزامياً لمرحلة التجربة الحالية
سياسة كلمة المرور بالطول فقط (٨ أحرف)	توصية	كافية لمرحلة التجربة، يُفضّل تعزيزها قبل الإطلاق الواسع
لا توجد نسخ احتياطي تلقائي لقاعدة البيانات	متطلب تشغيلي	يجب إعداده على مستوى بيئة النشر، وليس داخل الكود

الخلاصة

لم يتم العثور على أي عائق أمني على مستوى الكود يمنع الانتقال للتشغيل – كل ما تبقى هو قرارات نشر وربط خدمات خارجية (مزود إشعارات حقيقي، شهادة TLS، نسخ احتياطي).

ملاحظات ونصائح مهمة

أسئلة وحلول متكررة

ظهور خطأ SQLITE_CONSTRAINT_UNIQUE عند التعبئة

يحدث عادة عند تنفيذ `db:seed` على قاعدة بيانات مُعبأة مسبقاً. الحل: احذف ملفات قاعدة البيانات المحلية (`.db` و `.db-shm` و `.db-wal`) ثم أعد `npm run db:reset`.

المنفذ (Port) مستخدم مسبقاً عند التشغيل

أوقف أي عملية Node.js عالقة من تشغيل سابق قبل إعادة المحاولة، أو غيّر رقم المنفذ عبر متغير البيئة `PORT`.

ميزات الذكاء الاصطناعي لا تعمل

تعمل المنصة بوضع محاكاة (mock) تلقائياً دون مفتاح Gemini حقيقي، وتُعرض نتائج واقعية توضيحية. لتفعيل الذكاء الاصطناعي الحقيقي، أضف `GEMINI_API_KEY` في ملف `.env` واضبط `AI_MODE="gemini"`.

الفرق بين "الحافلات/الطلاب" في النظامين

يحتوي المشروع على جدولين منفصلين بنفس الاسم تقريباً (نظام قديم ونظام محوكم) – هذا تصميم متعمد لفصل الأنظمة، وليس خطأ مضاعفاً في البيانات.

جهات الاتصال والمرجعية التقنية

لأي استفسار تقني إضافي، يُرجى الرجوع إلى ملفات التوثيق الداخلية في مجلدي `server/` و `database/`، حيث تحتوي كل خدمة رئيسية على تعليقات توضيحية مفصلة لقرارات التصميم.

مَسَارًا – MASARA · منصة النقل المدرسي الذكية المحكومة

هذا الدليل تم إعداده كمرجع تعريفي وتقني شامل للمشروع بحالته الحالية.